RECETARIO WEB

Proyecto de Desarrollo Web — Entrega 2

Núcleo funcional: autenticación, roles, Docker y despliegue

INTEGRANTES

Angely Liced Herrera Roldán

Juan Camilo Barragán Pottes

GRUPO 500

07/11/2026

1. PROPÓSITO

Este documento presenta la evolución del proyecto Recetario Web y reúne los requisitos, características y decisiones técnicas que se han definido durante su desarrollo.

En la primera entrega se establecieron las bases del sistema, incluyendo la organización de las recetas, las opciones de votación y la arquitectura inicial. Para esta segunda entrega, el proyecto avanza con la incorporación del registro de usuarios, el inicio de sesión y la asignación de roles, con el propósito de controlar el acceso a las diferentes funcionalidades.

También se documentan los cambios realizados en la estructura de la aplicación, el uso de Docker y el proceso de despliegue inicial. De esta manera, el documento permite consultar tanto lo desarrollado como las características que se tienen previstas para las siguientes etapas.

1.1. ALCANCE DEL DOCUMENTO

Este documento describe el sistema Recetario Web y reúne la información correspondiente a su desarrollo en las diferentes entregas. Incluye el problema que se busca resolver, los objetivos, el alcance, los requisitos funcionales y no funcionales, la arquitectura, el modelo de datos, las tecnologías utilizadas y los diagramas UML que representan el funcionamiento y la estructura del sistema.

También presenta los avances realizados en la Entrega 1 y la Entrega 2, diferenciando las funcionalidades implementadas de aquellas que se encuentran previstas para las siguientes etapas.

1.2. DEFINICIONES

Término	Definición
Receta	Preparación con nombre, categoría, ingredientes, pasos y usuario dueño.
Opción	Alternativa votable asociada a una receta.
Usuario autenticado	Cuenta registrada con rol Usuario; puede gestionar sus recetas y votar.
Administrador	Usuario staff/superuser; accede a Django Admin y gestiona cualquier receta.
CRUD	Crear, consultar, actualizar y eliminar.
SRS	Software Requirements Specification.
Docker	Contenedor que empaqueta la aplicación Django para ejecución reproducible.

1.3. REFERENCIAS

- Documentación oficial de Django 5.1.
- IEEE Std 830-1998 (organización del SRS).
- Código fuente: carpetas recetario/ y recetas/.
- Repositorio: https://github.com/juancamilop1/Recetario-web
- Documento de Entrega 1 (base del presente SRS).

2. PROBLEMA A RESOLVER

En muchas ocasiones, las recetas de cocina se guardan en diferentes lugares, como cuadernos, conversaciones de WhatsApp o archivos personales. Esto puede dificultar su organización, hacer que se pierda información o generar varias versiones de una misma receta.

A partir de esta situación surge Recetario Web, una aplicación que busca reunir las recetas en un solo lugar para facilitar su registro, consulta y actualización. Además, permite votar por diferentes opciones y consultar los resultados para conocer las preferencias de los usuarios.

Durante la primera entrega, las operaciones principales de la aplicación estaban disponibles sin necesidad de iniciar sesión. Para la segunda entrega se incorpora un sistema de autenticación y roles, con el que cada usuario puede administrar sus propias recetas, mientras que el administrador conserva el acceso general para gestionar la información.

3. JUSTIFICACIÓN

El desarrollo de Recetario Web busca ofrecer una forma más organizada y sencilla de almacenar y consultar recetas de cocina. Al centralizar esta información, los usuarios pueden acceder a las preparaciones desde una misma plataforma y mantener sus datos actualizados.

Con los avances de la segunda entrega, también se hace necesario mejorar la forma en que se administra la información. Por esta razón, se incorpora el registro de usuarios y el inicio de sesión, junto con roles y permisos que permiten controlar quién puede crear, modificar o eliminar las recetas.

Por otra parte, el uso de Docker permite preparar la aplicación para ejecutarse en diferentes entornos con una configuración más controlada. Esto facilita su puesta en marcha y sirve como base para el despliegue del proyecto.

Se mantiene Django como framework principal porque ofrece herramientas para gestionar usuarios, trabajar con bases de datos y construir las diferentes funcionalidades de la aplicación sin necesidad de utilizar tecnologías adicionales para cada operación.

4. OBJETIVOS

4.1. OBJETIVO GENERAL

Desarrollar el núcleo funcional de Recetario Web, incorporando mecanismos de autenticación y control de acceso, junto con la actualización de la documentación, la preparación de la aplicación mediante Docker y su despliegue inicial

4.2. OBJETIVOS ESPECÍFICOS

- Incorporar el registro de usuarios y las opciones para iniciar y cerrar sesión.
- Establecer los roles de Usuario y Administrador, definiendo los permisos correspondientes para cada uno.
- Asociar las recetas con el usuario que las registra, permitiendo que cada persona gestione su propia información.
- Actualizar el documento SRS de acuerdo con los cambios realizados en la aplicación.
- Elaborar una primera versión del diagrama de secuencia para representar el funcionamiento de los procesos principales.

- Preparar un Dockerfile que permita ejecutar la aplicación Django dentro de un contenedor.
- Realizar el despliegue inicial de la aplicación y documentar su dirección de acceso pública.
- Preparar el video de presentación correspondiente a la segunda entrega.

4.3. PLAN DE TRABAJO POR ENTREGAS

Fase	Objetivo	Estado
Entrega 1	SRS, UML, backlog, repo, video inicial	Completada
Entrega 2	Auth, roles, secuencia, Docker, deploy, SRS+trazabilidad, video	En curso
Entrega 3	Node.js, Swagger, IA, Compose	Pendiente
Definitiva	SRS final y video de 10 min	Pendiente

5. ALCANCE DEL PROYECTO

5.1. Funcionalidades de Entrega 1

ID	Funcionalidad
F-01	Listar recetas
F-02	Ver detalle con ingredientes, preparación, opciones
F-03	Registrar recetas
F-04	Editar recetas
F-05	Eliminar recetas con confirmación
F-06	Votar opciones
F-07	Consultar resultados de votos
F-08	Gestionar datos en Django Admin

5.2. Funcionalidades añadidas en Entrega 2

ID	Funcionalidad
F-10	Registro de usuarios
F-11	Login y logout
F-12	Roles: Usuario autenticado y Administrador
F-13	CRUD restringido: solo dueño o admin editan/eliminan
F-14	Receta asociada a usuario
F-15	Diagrama de secuencia
F-16	Dockerfile / contenedor Django funcional
F-17	Despliegue inicial
F-18	SRS actualizado + matriz de trazabilidad + video 2

5.3. revistas para Entrega 3 y definitiva

Para las próximas etapas del proyecto se contempla ampliar la aplicación mediante la incorporación de un microservicio desarrollado en Node.js, documentación de servicios con Swagger/OpenAPI y comunicación entre Django y Node.js. También se plantea integrar funcionalidades basadas en inteligencia artificial y utilizar Docker Compose para facilitar la ejecución conjunta de los diferentes servicios.

5.4. Fuera de alcance

Por el momento, el proyecto no contempla el desarrollo de una aplicación móvil nativa, el envío de notificaciones push ni la implementación de una infraestructura de distribución de imágenes a gran escala. Estas funcionalidades no hacen parte de los objetivos definidos para las entregas actuales.

6. DESCRIPCIÓN GENERAL DEL SISTEMA

Recetario Web está desarrollado como una aplicación web monolítica utilizando Django y el patrón de arquitectura MVT (Modelo, Vista y Plantilla).

El proyecto recetario contiene la configuración general de la aplicación, mientras que la aplicación recetas reúne la lógica relacionada con la gestión de recetas, las vistas y las funcionalidades de autenticación y control de acceso de los usuarios.

Para almacenar la información se utiliza SQLite. Los archivos estáticos se gestionan mediante WhiteNoise en el entorno de despliegue con Docker.

6.1. Tipos de usuarios

El sistema contempla los siguientes tipos de usuarios:

- **Usuario anónimo:** puede consultar el listado de recetas, ver sus detalles y revisar los resultados de las votaciones. No puede crear, editar ni eliminar recetas, ni participar en las votaciones.
- **Usuario registrado:** puede iniciar y cerrar sesión, crear recetas, editar y eliminar sus propias recetas y participar en las votaciones.
- **Administrador:** cuenta con permisos de administración para gestionar la información del sistema mediante Django Admin.

7. REQUISITOS ESPECÍFICOS

7.1. Requisitos funcionales

ID	Descripción	Prioridad
RF-01	Listar recetas y acceder al detalle	Alta
RF-02	Mostrar nombre, categoría, ingredientes y preparación	Alta
RF-03	Crear recetas (usuario autenticado)	Alta
RF-04	Editar recetas propias o como admin	Alta
RF-05	Eliminar con confirmación (dueño o admin)	Alta
RF-06	Votar opción (usuario autenticado)	Media
RF-07	Mostrar conteo de votos	Media
RF-08	Validar selección antes de votar	Media
RF-09	Admin gestiona Receta y Opción	Alta
RF-10	Registro de cuenta	Alta
RF-11	Login / logout	Alta
RF-12	Asignar dueño al crear receta	Alta
RF-13	Ejecutar la app en contenedor Docker	Alta
RF-14	Publicar instancia accesible remotamente	Alta

7.2. Requisitos no funcionales

ID	Descripción	Categoría
RNF-01	Operaciones locales típicas < 2 s	Rendimiento

RNF-02	CSRF en formularios POST	Seguridad
RNF-03	Código en GitHub	Mantenibilidad
RNF-04	Configuración sensible por variables de entorno	Seguridad
RNF-05	Documentación E2 en PDF dentro de docs/	Documentación
RNF-06	Imagen Docker reproducible con requirements.txt	Portabilidad

7.3. Modelo de datos

El sistema utiliza las siguientes entidades principales:

- **Receta:** almacena la información de cada receta, incluyendo su identificador, nombre, categoría, ingredientes y preparación. También cuenta con una relación con el usuario que la creó mediante una clave foránea hacia el modelo User de Django. Esta relación permite identificar al propietario de cada receta. El campo puede admitir valores nulos para conservar registros creados antes de implementar la autenticación.
- **Opción:** representa las opciones disponibles en la votación de una receta. Contiene un identificador, la relación con la receta correspondiente, el texto de la opción y la cantidad de votos registrados.
- **User:** corresponde al modelo de usuarios de Django y almacena la información de las cuentas registradas en el sistema.
- **Group:** permite organizar los usuarios mediante grupos de permisos. El sistema contempla el grupo Usuario y utiliza los permisos administrativos propios de Django para los administradores.

Las entidades Receta y Opción se relacionan mediante una clave foránea, de manera que una receta puede tener varias opciones de votación.

8. TECNOLOGÍAS UTILIZADAS

Componente	**Tecnología**
Lenguaje	Python 3.12
Framework	Django 5.1
Auth	django.contrib.auth + grupos
BD (E2)	SQLite
Servidor contenedor	Gunicorn + WhiteNoise
Contenedores	Docker
Versionado	Git + GitHub

9. ARQUITECTURA DEL SISTEMA

Recetario Web utiliza una arquitectura monolítica basada en el patrón MVT de Django. Esto significa que las funcionalidades principales de la aplicación se encuentran organizadas dentro de un mismo proyecto, separando las responsabilidades entre las vistas, los modelos y las plantillas.

El usuario interactúa con la aplicación desde el navegador. Las solicitudes son recibidas por las rutas de Django y procesadas por las vistas, que se encargan de ejecutar las operaciones correspondientes, validar los permisos y comunicarse con los modelos para consultar o guardar información en la base de datos SQLite.

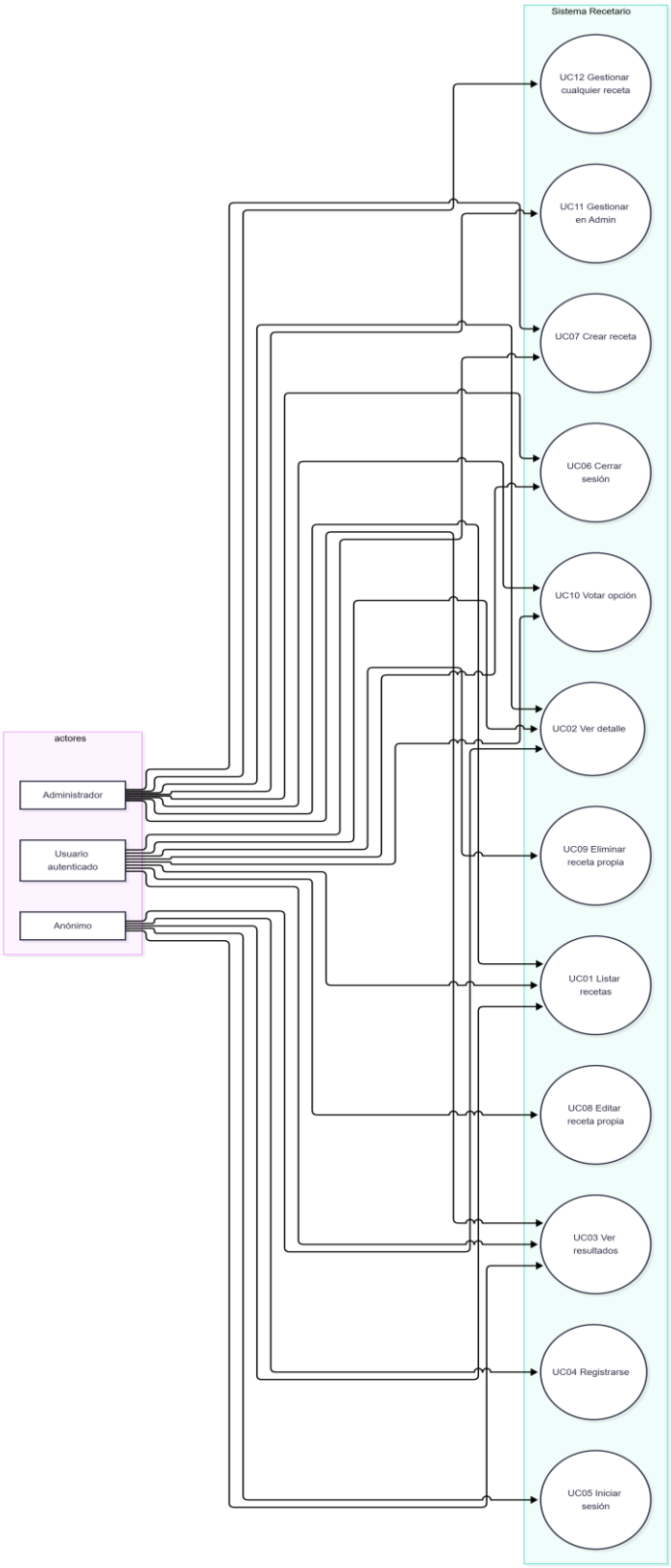
Con los cambios de la segunda entrega, la aplicación incorpora el sistema de autenticación de Django para gestionar las sesiones de los usuarios y controlar el acceso según sus permisos. De esta forma, las operaciones de creación, edición y eliminación de recetas quedan restringidas al propietario de la información o al administrador.

Para preparar la ejecución de la aplicación en un entorno de contenedores, se utiliza Docker junto con Gunicorn como servidor de aplicaciones y WhiteNoise para la gestión de archivos estáticos.

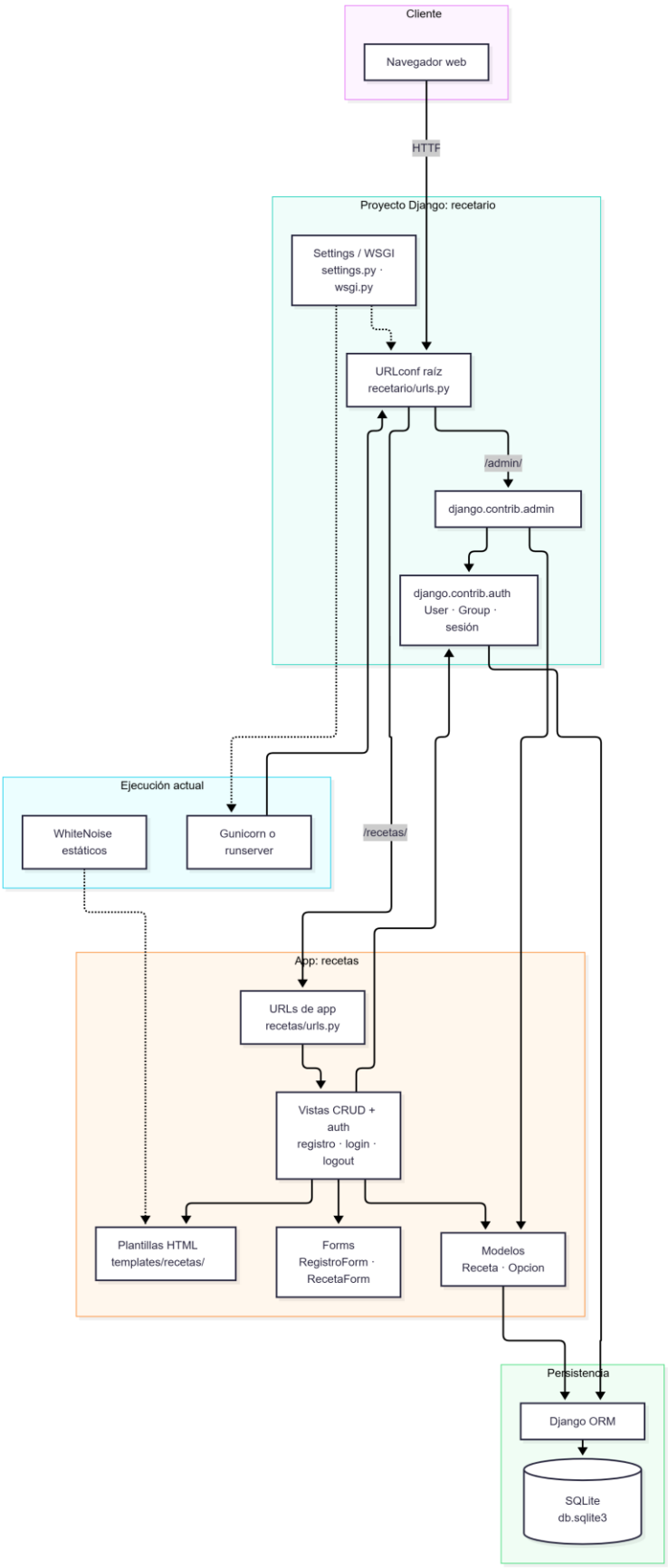
La arquitectura actual mantiene a Django como componente principal. Para las siguientes etapas se contempla ampliar la solución mediante servicios desarrollados en Node.js e integrar nuevas funcionalidades, incluyendo inteligencia artificial. Estas mejoras forman parte de la evolución prevista del proyecto y no deben confundirse con los componentes que ya están implementados.

10. DIAGRAMAS UML

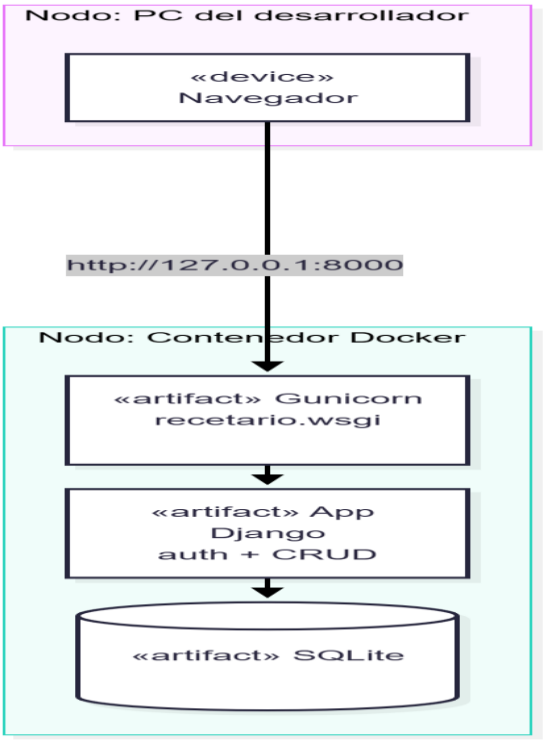
10.1. Casos de uso



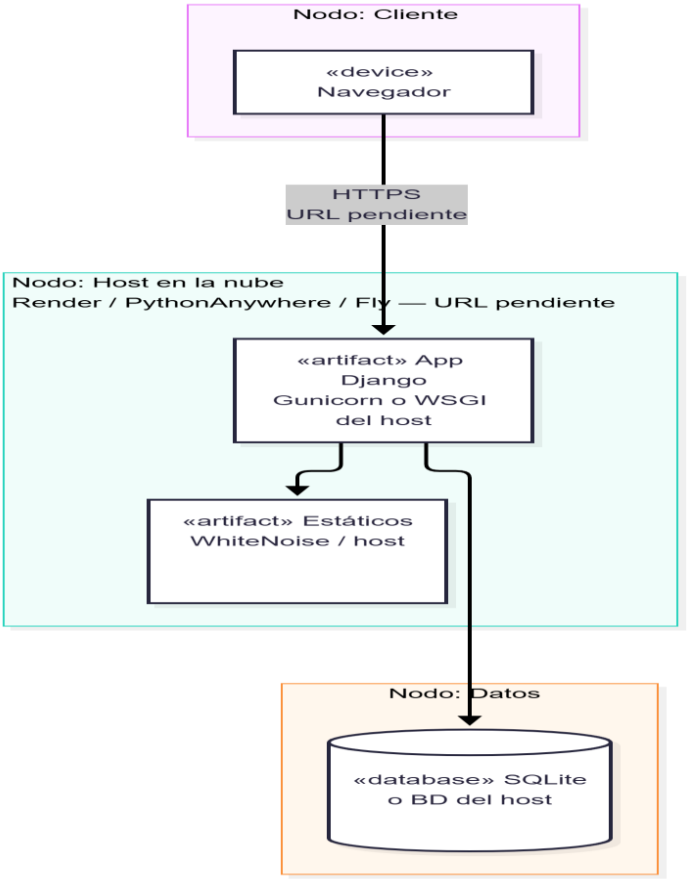
10.2. Componentes



10.3. Despliegue con docker



10.4. Despliegue publicado



10.5. Diagrama de secuencia (primera versión)

Emisor	Receptor	Mensaje
Usuario	Navegador	Ingresa a /recetas/registro/.
Navegador	Vista de registro	Envía una solicitud GET para cargar el formulario.
Vista de registro	Navegador	Devuelve el formulario de registro.
Usuario	Navegador	Completa y envía los datos del registro.
Navegador	Vista de registro	Envía una solicitud POST con las credenciales.
Vista de registro	Modelo User	Valida los datos y solicita la creación de la cuenta.
Modelo User	Base de datos	Guarda la información del nuevo usuario.
Vista de registro	Sistema de autenticación	Inicia la sesión del usuario y lo redirige al listado de recetas.
Usuario	Navegador	Ingresa a la opción para crear una receta.
Navegador	Vista de creación de receta	Envía una solicitud GET a /recetas/crear/.

Participantes: Usuario, Navegador, Vista registro, Vista crear_receta, Modelos User/Receta, SQLite. Se puede complementar con captura gráfica exportada desde Mermaid.

11.BACKLOG

ID	Épica	Fase	Estado
E1	Gestión de recetas	E1	Hecho
E2	Votación	E1	Hecho
E3	Administración	E1–E2	Hecho
E4	Calidad / docs	E1–E2	Hecho
E5	Autenticación y roles	E2	Hecho
E6	Contenedores y despliegue	E2	Hecho
E7	Microservicios e IA	E3	Pendiente

11.1. Historias Entrega 2 (Must have)

ID	Historia	Estado
HU-10	Como visitante, quiero registrarme e iniciar sesión.	Hecho
HU-19	Como sistema, quiero roles Usuario/Admin con permisos.	Hecho
HU-20	Como equipo, queremos diagrama de secuencia.	Hecho (doc)
HU-21	Como equipo, queremos Dockerfile funcional.	Hecho
HU-22	Como equipo, queremos deploy inicial	Hecho — https://recetario-web-

	publicado.	pilj.onrender.com/recetas/
HU-23	Como equipo, queremos SRS + matriz de trazabilidad.	Hecho

12.MATRIZ DE TRAZABILIDAD

Req / HU	Descripción	Componente	Épica	Entrega
RF-01 / HU-01	Listar recetas	lista_recetas	E1	1
RF-02 / HU-02	Detalle	detalle_receta	E1	1
RF-03 / HU-03	Crear	crear_receta + RecetaForm	E1/E5	1–2
RF-04 / HU-04	Editar	editar_receta + permiso dueño	E5	2
RF-05 / HU-05	Eliminar	eliminar_receta + permiso	E5	2
RF-06 / HU-06	Votar	votar (login_required)	E2/E5	2
RF-07 / HU-07	Resultados	resultados	E2	1
RF-09 / HU-08	Admin	RecetaAdmin / OpcionAdmin	E3	1–2
RF-10 / HU-10	Registro	registro + RegistroForm	E5	2
RF-11	Login/Logout	RecetarioLoginView / LogoutView	E5	2
RF-12	Dueño receta	Receta.usuario	E5	2
RF-13 / HU-21	Docker	Dockerfile + gunicorn	E6	2
RF-14 / HU-22	Deploy	Host público (URL)	E6	2
HU-20	Secuencia	§10.4 documento	E6	2
HU-23	Trazabilidad	§12 documento	E4	2

13.DOCKER Y DESPLIEGUE

El proyecto cuenta con un archivo Dockerfile basado en Python 3.12-slim, que permite preparar el entorno necesario para ejecutar la aplicación. Durante el proceso de construcción se instalan las dependencias del proyecto y se recopilan los archivos estáticos mediante collectstatic.

Al iniciar el contenedor se ejecutan las migraciones de la base de datos y se levanta la aplicación mediante Gunicorn, utilizando el puerto 8000.

Para su configuración se contemplan las siguientes variables de entorno:

- DJANGO_SECRET_KEY: clave utilizada por Django.
- DJANGO_DEBUG: permite activar o desactivar el modo de depuración.
- DJANGO_ALLOWED_HOSTS: define los dominios o direcciones permitidos.
- DJANGO_CSRF_TRUSTED_ORIGINS: especifica los orígenes de confianza para las solicitudes protegidas por CSRF.

Ejecución local con Docker

Para construir la imagen se utiliza:

docker build -t recetario-web .

Para ejecutar el contenedor localmente:

docker run --rm -p 8000:8000 -e DJANGO_ALLOWED_HOSTS="*" recetario-web

Una vez iniciado, el sistema puede consultarse desde http://localhost:8000.

Despliegue

La aplicación está publicada en Render. URL pública: https://recetario-web-pilj.onrender.com/recetas/

El servicio se construye desde el repositorio GitHub mediante el Dockerfile del proyecto (Gunicorn en el puerto 8000) y queda accesible por HTTPS.

14.HISTORIAL DEL DOCUMENTO

Versión	Fecha	Descripción
1.0	04/11/2026	Entrega 1 — definición y arquitectura
2.0	09/11/2026	Entrega 2 — auth, roles, secuencia, Docker, trazabilidad